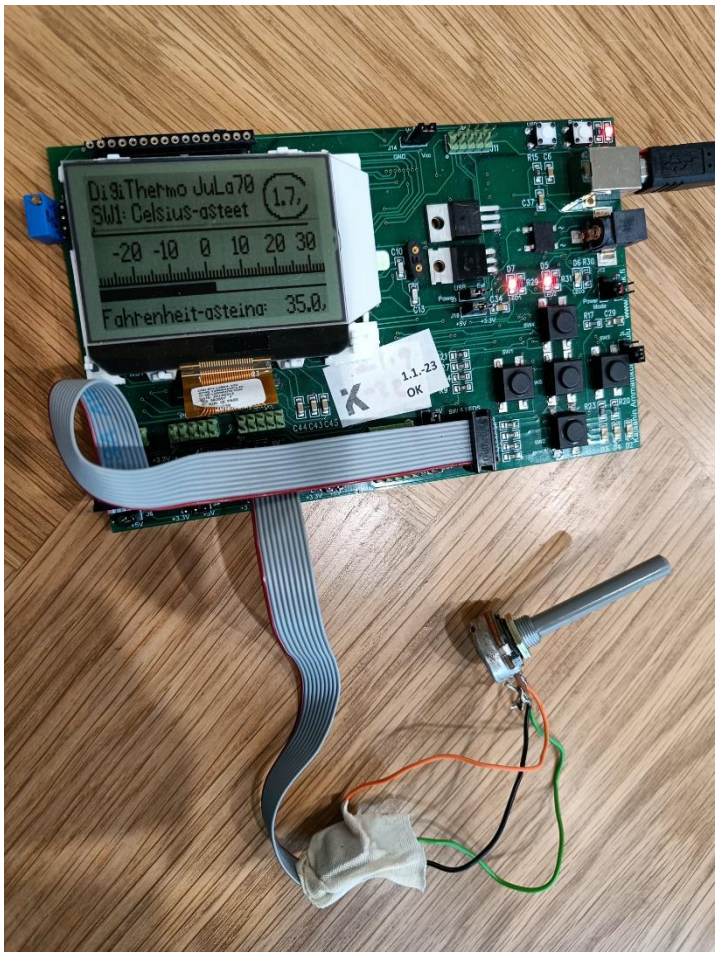
Juha Laurila

Mikrokontrolleritekniikka 1

Projektityö



Insinööri

Tieto- ja viestintätekniikka

Kevät 2023

KAMK • University of Applied Sciences

Sisällys

1 Johdanto 1

2 Suunnitelma 2

2.1 Ajankäyttösuunnitelma 2

2.2 Kytkentä 2

2.3 Ohjelman suunnitelma sekä sanallisena että vuokaaviona (ohjelman logiikka) 4

3 Työn toteutus 6

4 Yhteenveto 9

1 Johdanto

Projektityön päämääränä oli kehittää digitaalinen lämpömittari, joka imitoi grafiikaltaan ja lämpötilan näyttämistavaltaan suurelta osin analogista, perinteistä nestepilarin lämpölaajenemiseen perustuvaa mittaria. Kehitys- ja toteutusalustana oli Kamkin Atmega 128 mikrokontrolleriin perustuva opetuskortti, jossa Batron 128 x 64 pix LCD-näyttö. Todellista lämpötila-anturia ei ollut vaan sen asemesta käytettiin 10 kOhm potentiometriä erilaisten lämpötila-arvojen simuloimiseen. Ohjelmointi tapahtui Avr Studio 4 kehitysympäristössä C-kieltä käyttäen. Keskeisiä vaiheita projektissa oli graafiselle näytölle rakennettavien lämpömittarin graafisten kiinteiden elementtien ohjelmointi, kytkimen painalluksen ja lämpötilan mukaan muuttuvien elementtien ohjelmointi, itse kytkimen ja opetuskortin ledien ohjelmointi, opetuskortin AC-muuntimen käyttöönotto sekä potentiometrin fyysinen kytkeminen opetuskorttiin ja ohjelmallinen käyttöönotto.

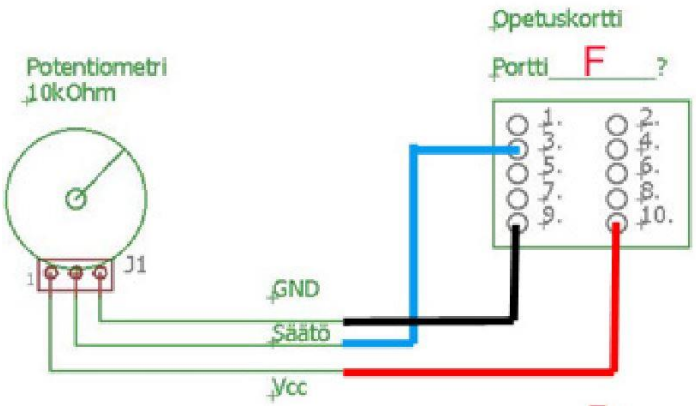
2 Suunnitelma

2.1 Ajankäyttösuunnitelma

Erittäin tarkkaa ajankäyttösuunnitelmaa en laatinut. Kurssin aiemmat harjoitustehtävät antoivat kuitenkin riittävän varmuuden siitä, että arvelin suoriutuvani projektista melko lyhyessä ajassa. Jäsentelin aluksi mielessäni kokonaisuutta ennen projektin alkua ja ajankäyttö jakaantuisi kahteen kokonaisuuteen: mittarinäytön grafiikan suunnitteluun ja toteutukseen sekä lämpömittarin toiminnallisuuksien suunnitteluun ja toteutukseen. Toki asiat menivät osin limittäin miettiessä mitkä graafiset elementit ovat lämpötilan mukaan muuttuvia ja miten se pitää ottaa huomioon näkymän suunnittelussa. Itse grafiikan ohjelmointiin meni kokonaisajan alusta noin kolmasosa, loppuaika oli toiminnallisuuksien implementointia ja testailuja. Toki kun olen töissä, oli päiviä, jolloin en ehtinyt paneutumaan asiaan ollenkaan. Sain tyydyttävästi toimivan version toteutettua n. 2 viikossa.

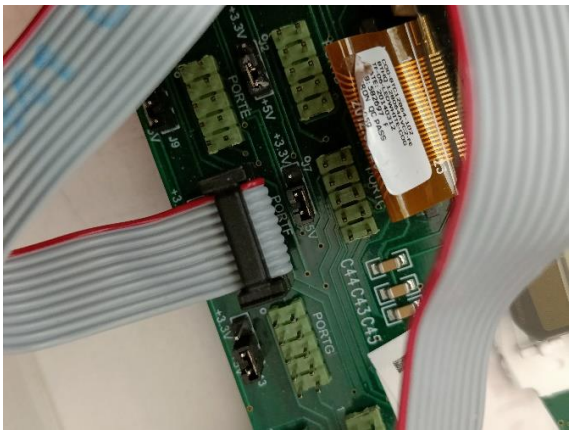
2.2 Kytkentä

Koska projektissa käytettiin varsinaisen lämpötila-anturin sijaan potentiometriä, ei erillistä fyysistä sovituskytkentää anturille tarvittu. Käytetty 10 kOhm potentiometri kytkettiin opetuskortin F-porttiin (analogiatuloon) lattakaapelilla seuraavasti:

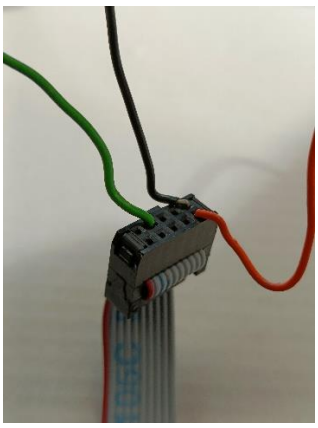


Kuva 1. Potentiometrin kytkentäperiaate opetuskortin F-porttiin. Ohjelmallisesti portin numerot alkavat bitti 0:sta (0-9) joten kuvaan merkitty 1 edustaa bittiä 0. Kun tulin käyttämään työssä AD-muuntimen sisääntulona input 2:sta (2. bitti kirjoitettu AD-muuntimen ADMUX-rekisterissä

ykköseksi), on potentiometrin säätöjohdin (input) siten kytkettävä kuvan mukaisesti liittimen 3. reikään. Vcc on bitti 9 (liittimen reikä 10) ja GND edellinen bitti 8 (liittimen reikä 9).



Kuvat 2 ja 3. Lattakaapelin liittäminen kortin F-porttiin. Lattakaapelin toinen pää (liitin) painettiin ensin opetuskortin F-porttiin niin päin, että portin 0-merkintä ja lattakaapelin punainen johdin olivat kohdakkain.



Kuva 4. Lattakaapelin ja potentiometrin johtimien liittäminen. Vihreä=input.

2.3 Ohjelman suunnitelma sekä sanallisena että vuokaaviona (ohjelman logiikka)

Kun ohjelmaa ajetaan, linkitetään pääohjelmaan ensin tarvittavat otsikko- ja kirjastotiedostot. Seuraavaksi määritellään väylään kytkettyjen laitteiden osoitteet, portteihin kytketyt laitteet ja LEDit. Tämän jälkeen esitellään tarvittavat globaalit muuttujat sekä esitellään void ADC_init() -funktio. Tämän jälkeen alkaa main() -funktio, jonka aluksi esitellään tarvittavat muuttujat, joita tarvitaan numeeristen muuttuvien lämpötila-arvojen luomisessa sekä lämpötilapalkin luomisessa. Tämän jälkeen kutsutaan ad-muuntimen alustajafunktiota ADC-init() sekä asetetaan kytkin 1 ja 2 sisääntuloksi sekä otetaan LED:it käyttöön. Seuraavaksi esitellään merkkipuskuri sekä ympyrän piirtämisessä, asteikon viivojen luomisessa tarvittavat muuttujat. Tämän jälkeen suoritetaan ensiksi LCD-näytön alustus ja sen jälkeen käyttöliittymän piirtämisrutiinit.

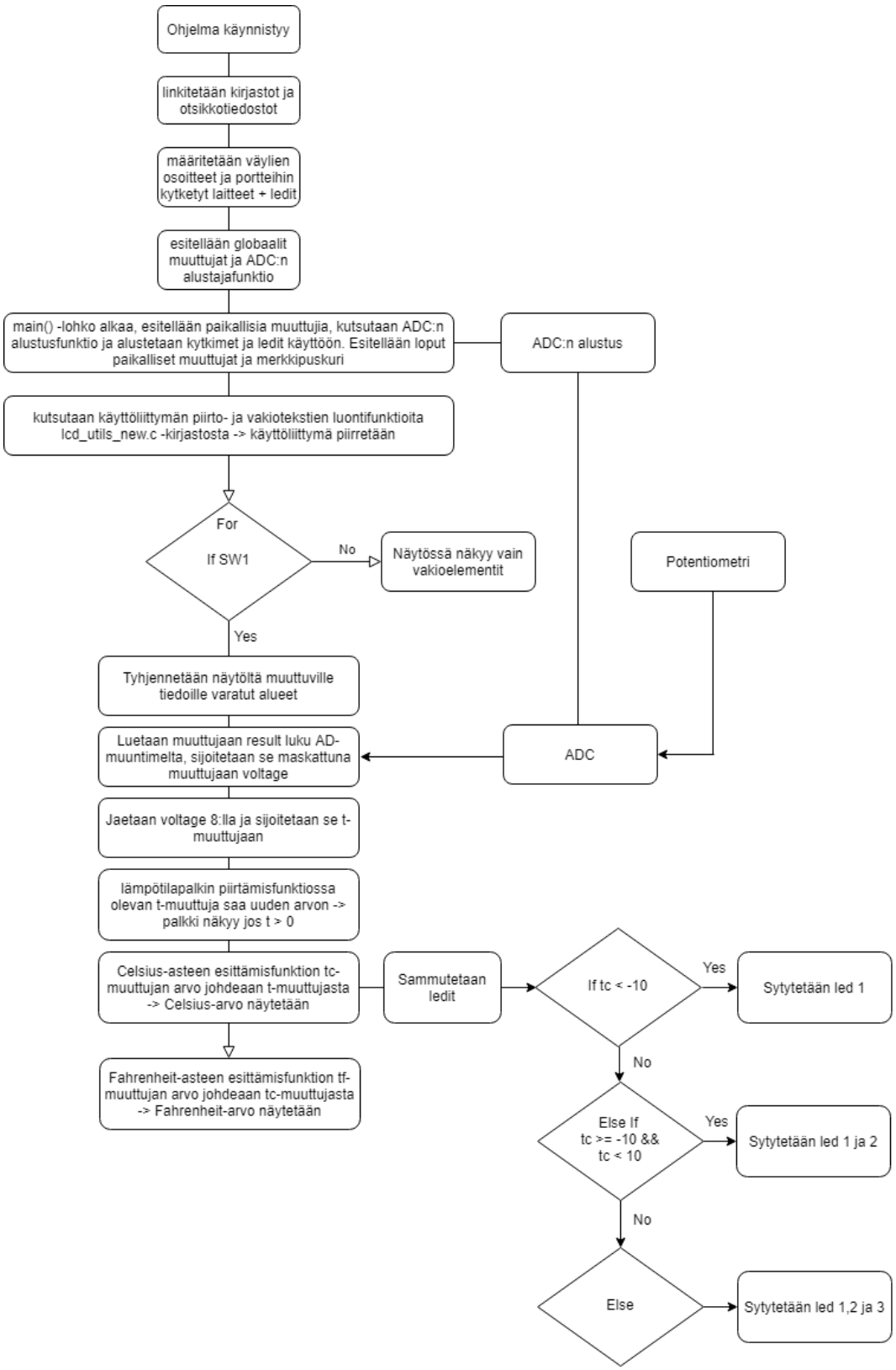
Aluksi piirretään kiinteät tekstielementit, vaakaviivat sekä asteikon pystyviivat, ympyrä, jonka sisään tulee celsius-lukema sekä suorakaide, joka kuvastaa lämpötilapalkkia. Elementit piirretään kutsumalla lcd_utils_new.c -tiedoston valmiita funktioita ja asettamalla niiden parametriksi sopivia arvoja elementtien sijainnin, sisällön ja ulkoasun luomiseksi. Lopuksi kutsutaan (piirtämisfunktioiden jälkeen) LCD_Display() -funktiota , joka näyttää luodut elementit näytöllä. Tämän jälkeen käyttöliittymä on valmis ja siinä on kaikki kiinteät elementit näkyvissä. Suorakaidepalkki ei vielä piirry näkyviin, koska sen toinen x-parametri – palkin oikea reuna on toteutettu muuttujalla, jonka arvo on vielä nolla.

Tämän jälkeen ohjelman loppuosa toimii for-silmukassa. Silmukan sisällä on toiminnallisuus, joka käynnistyy käyttäjän painaessa SW1-kytkintä. If-ehtolauseen sisälle on laitettu ehto, joka tutkii, onko SW1 painettu. Mikäli on painettu, tyhjennetään näytöltä ensin muuttuvien arvojen alueilta näyttö. Tämän jälkeen luetaan muuttujaan result AD-muuntimen arvo, joka riippuu kehitysalustaan portti F:een liitetyn potentiometrin antamasta jännitteestä (joka 10 bitillä maskattuna on arvo välillä 0-1023). Tämä maskattu arvo sijoitetaan voltage-muuttujaan (muuttujan nimi voisi olla kuvaavampi, jännitteestähän ei ole kyse).

Tämän jälkeen voltage jaetaan 8:lla, josta saadaan näytön leveyden sisältävät pikselit (128), saatu tulos sijoitetaan muuttujaan t, jota käytetään lämpötilapalkin oikean reunan sijainnin määritykseen. Edelleen, muuttujasta t johdetaan muuttuja tc, jota käytetään celsius-asteiden numeeriseen esittämiseen. Fahrenheit-luku saadaan muuttujasta tc celsius-fahrenheit kaavan avulla, ja se näytetään niin ikään SW1:n painalluksen jälkeen.

Kehitysalustan omat ledit näytetään myös SW1:n jälkeen seuraavasti: alle -10 C asteessa palaa yksi LED, 10 C asteessa tai korkeammissa lämpötiloissa palaa kolme LEDiä, muulloin palaa kaksi LEDiä.

Ohjelman logiikka vuokaaviona:



3 Työn toteutus

Kuten aiemmin viittasin, oli ensimmäinen puolisko projektia käyttöliittymän suunnittelua ja toteutusta ja hiukan myös toiminnallisuuksien toteutuksen alustavaa pohdintaa. Hetken mietittyäni päätin, että teen mittarin vaakasuoraan asentoon, koska se on järkevintä näytön muoto huomioiden. Mietin normaalia ulkolämpömittaria, asteet menevät nollan molemmin puolin muutaman kymmenen asteen positiiviseen ja negatiiviseen suuntaan. Päätin toteuttaa sellaisen ja aloitin.

Olin jo aiemmassa harjoitustehtävässä perehtynyt aika paljon ko. tehtävän mukana annetun lcd_utils_new.c -kirjaston ominaisuuksiin ja niinpä minulla oli suurin piirtein funktiot tiedossa käyttöliittymän toteutukseen. Aivan ensiksi päätin, että mittarillani olisi oltava jokin nimi. LCD_PrintString() -funktiota hyödyntäen sijoitin nimitekstin vasempaan yläkulmaan antamalla funktion parametreille sopivat x:n ja y:n arvot. Samalla tavalla loin myöhemmin myös muut tekstisisällöt käyttöliittymään.

Seuraavaksi tein näytön oikeaan yläkulmaan ympyrän, jonka sisään tulisi lämpötila numeerisesti Celsius-asteina. Toteutin tämän vastaavalla tavalla kuin aiemmassa harjoitustyössä, LCD_DrawPixel() funktiota ja ympyrän yhtälöä hyödyntäen.

Seuraavaksi loin vaakaviivoja; ensiksi ylimmäisen vaakaviivan yläosan nimen ja ympyrän alapuolelle käyttäen LCD_DrawLine()-funktiota. Näin sain jaettua yläosan alaosaan tulevasta analogista mittaria imitoivasta osasta. Samoin tein myös toisen vaakaviivan riittävästi alemmaksi kuvaamaan analogisen mittarin ”lasiputken” toista (ylempää) reunaa. Tästä viivatasosta ylöspäin ajattelin toteuttaa mitta-asteikon numeroilla varustettuna. Samalla tein vielä yhden viivan hieman alemmaksi, siitä tulisi ”lasiputken” alempi reuna. Väliin pitäisi tehdä sitten lämpötila-arvoa kuvaava palkki.

Toteutin sitä ennen kuitenkin mitta-asteikon ja sen yläpuoliset numerot. Tein ensin mitta-asteikkoon asteiden välit lyhyellä pystysuuntaisella viivalla monistettuna for-silmukassa. Tein myös viiden asteen välit hieman pidemmällä viivalla samalla periaatteella, kuin myös 10 asteen välit. Kun mitta-asteikko näytti hyvältä, aloin sijoittamaan numeroita asteikon yläpuolelle LCD_PrintString() -funktiota käyttäen. Jotta sain numerot täsmällisesti viivojen kohdille, vaati se hieman numerokohtaista säätöä koodissa.

Nyt mitta-asteikon alla olevaan ”lasiputkeen” piti kehittää ”neste”. lcd_utils_new.c -kirjastossa oleva 4:n parametrin LCD_FillRect() olisi siihen nappiratkaisu. Sijoitin funktiolla saatavan suorakaiteen sopimaan ”lasiputken” sisään ja alkamaan vasemmasta reunasta. Toinen x arvo – suorakaiteen oikea reuna, piti vaan korvata muuttujalla, johon tulisi ”lämpötilan” mukaan vaihtuva tieto potentiometriltä ADC:n kautta. Testailin ensin funktion ja näkymän toimivuutta muuttelemalla muuttujan arvoja ”kovakoodatusti”, ja totesin että periaatteessa toimii. Palasin vielä aiemmin tekemäni ympyrän pariin ja toteutin sen sisään vielä kiinteästi pari merkkiä; tähän ajattelin myöhemmin sijoittavani numeerisen lämpötila-arvon.

Tässä vaiheessa oli suurin työ käyttöliittymään jo tehty. Olikin aika ruveta miettimään, miten tieto saataisiin potentiometriltä muuttamaan suorakaidepalkkia – ”nestettä lasiputkessa”. Olikin aika ruveta ottamaan AD-muunninta käyttöön. Implementoin AD-muuntimen aiemman AD-harjoituksen mukaisesti. ADMUX rekisteriin kirjoitettiin bitti 2 ykköseksi, jolloin AD-input-kanavana olisi 2. Samoin piti miettiä, miten suorakaiteen toiseen x-muuttujaan (oikea reuna) saadaan oikea tieto palkin liikuttamiseksi. Tämä vaati ohjelmallista sovitusta – AD-muunninhan muuttaa potentiometriltä tulevan analogisen jännitteen 1024:ksi erilliseksi diskreetiksi bittisanaksi, joita taas voi hyödyntää eri lämpötila-arvojen näyttämisen toteuttamiseksi. LCD-näytön leveys on pikseleissä 128. Jotta siihen sai erillisiä asteen välillä olevia viivoja, oli luonnollisesti jätettävä joka toinen pikseli leveyssuunnassa aktivoimatta. Niinpä asteikko oli 128/2 pikseliä eli 64 aktiivista (mustaa) pikseliä. Toisin sanoen, AD-muuntimen resoluutio – 1024 eri tasoa pitäisi sovittaa lämpötilamittarini mittausvälille, joka on 64 astetta = 128 pikseliä. Tämä toteutui kaavassa 1024/8=128.

Eli kun ADC:lta tulee (potentiometrin asennosta riippuen) jokin lukema välillä 0-1023, jaetaan se 8:lla, jolloin saatu tulos sijoitetaan muuttujaan t, joka taas sijoitetaan LCD_FillRect()-funktion kolmanteen parametriin, joka edelleen vaikuttaa suorakaiteen oikeaan reunaan x-akselilla. Näin palkki saadaan liikkumaan ”lämpötilan” mukaan.

Tässä vaiheessa olin siirtänyt lämpötilapalkin piirtämistoiminnallisuuden jo näppäinpainalluksen for-silmukkaan SW1 taakse, ja asettanut opetuskortin kytkimen (näppäimen) 1 sisääntuloksi. Sain toimimaan aina kerran, kun käänsin ja ajoin ohjelman. Ongelma oli, että palkki ei nollautunut eikä lämpötila muuttunut järkevästi, kun potentiometriä välillä pyöritti johonkin suuntaan. Niinpä lisäsin kirjastosta LCD_ClearArea() -funktion nollaamaan palkin lukeman aina kun SW1 on painettu, ja sen jälkeen vasta uudelleen piirto muuttuneella suorakaiteen oikean reunan muuttujan arvolla.

Kun ”lasiputkessa” oleva ”neste” käyttäytyi johdonmukaisesti – potentiometrin toinen ääripää näytti -32 ja toinen +32, oli aika viimeistellä mittari. Muutin ympyrän sisään sijoittamani parin merkin paikalle celsiuslukeman. Celsiuslukema tehtiin LCD_PrintString() -funktiolla, jossa on kolme parametriä, merkkipuskuri sekä x:n ja y:n arvo. Merkkipuskuria varten piti luoda uusi float-tyyppinen muuttuja, tc, joka kuvaa lämpötilaa lukuna. Muuttuja tc:n arvo saatiin t funktiosta seuraavasti: tc=t/2-30;
Näin astelukema saatiin muuttumaan juuri oikein potentiometrin mukaan ja stemmaamaan lämpötilapalkin näyttämän kanssa.

Lopuksi tein vielä kaksi lisäystä ”SW1 painettu” -for-silmukkaan. Lisäsin lämpötilalukeman näytön myös Fahrenheit-asteina ”analogiamittarin nesteputken” alle varustettuna ohjetekstillä. Sitä varten tarvitsin vielä yhden float-muuttujan, tf:n, johon sijoitin tc:n arvon muuttuna fahrenheit-asteiksi seuraavasti: tf= tc*1.8+32;

Viimeisin lisäys oli kortin omien ledien käyttö kolmen eri lämpötila-alueen indikoimiseen.
Otin LEDit koodissa käyttöön. Tein sitten ”SW1 painettu” -for-silmukkaan vielä if, if else, else -ehtorakenteen siten, että jos lämpötila-alue on alhaisempi kuin -10, palaa vain LED1, jos =>-10 mutta pienempi kuin 10, palaa LED1 ja LED2, muuten pala kolme lediä.

4 Yhteenveto

Lopuksi voin todeta, että projekti oli hyvin mielenkiintoinen ja opettava. Mittarisovellus vaikutti aluksi ehkä vähän yksinkertaiselta aiheelta projektiksi, mutta osoittautui kuitenkin mielekkääksi ja sopivaksi niin työmäärältään kuin haastavuudeltaan. Projekti oli varsin helppo palastella osakokonaisuuksiin, kuten käyttöliittymän – mittarin ulkoasun eri elementtien toteutukseen sekä yhtä lailla toiminnallisuuksien implementoinnin suhteen.

Projekti kokosi hyvin kurssilla opitut asiat ja antoi reilusti myös uutta haastetta. Kyseisen kaltainen mittarisovellus on helppo laajentaa tarvittaessa käsittämään myös muunlaisia mittauksia kuten kosteuden, äänen tai muun ilmiön mittauksiin. Kun sovellukseen ja korttiin alettaisiin liittämään todellisia antureita ja sensoreita, siirryttäisiin kehitystyössä jo todellisten laitteiden suunnitteluun potentiometrillä toteutetun simulaation sijaan.